Проект: Бот-органайзер «Седьмой квант»

Команда: Седьмой квант (ДТ «Кванториум» г. Саратов)
Направление: Подземелье знаний
Трек разработки: Docker

Команда:

Филина Екатерина Владимировна (преподаватель ДТ «Кванториум»)

Майборода Артём

Трепшин Иван

Андрей Базыгин

Ссылка на бота:

https://max.ru/ideya_fiks_183_bot

- **Репозиторий GitHub:**

  https://github.com/ekaterinafilinav/seventh-quant-bot/blob/main/README.md

- **Docker-образ:** `ekaterinafilina/seventh-quant-bot`

  docker run -e BOT_TOKEN=токен ekaterinafilina/seventh-quant-bot

1. Проблема

В учебных чатах сообщения с домашними заданиями (ДЗ), файлами и датами контрольных теряются среди флуда. Функции «Избранное» и «Сохранённые» превращаются в свалку. По результатам нашего опроса (42 респондента), 74% школьников теряли сообщение с ДЗ, а 53% пропускали олимпиады или конкурсы из-за потерянных сообщений.

2. Решение

Бот-органайзер, который превращает хаос учебных чатов в порядок. Пользователь пересылает боту сообщение или скриншот, а бот автоматически распознает текст, определяет тип задачи (ДЗ, олимпиада, встреча, дедлайн), выделяет дату и время, сохраняет задачу в базу данных и напоминает о ней в нужный момент.

Главное отличие от Notion или Todoist: **не нужно вручную переносить задачи**. Порядок создается прямо из потока сообщений.

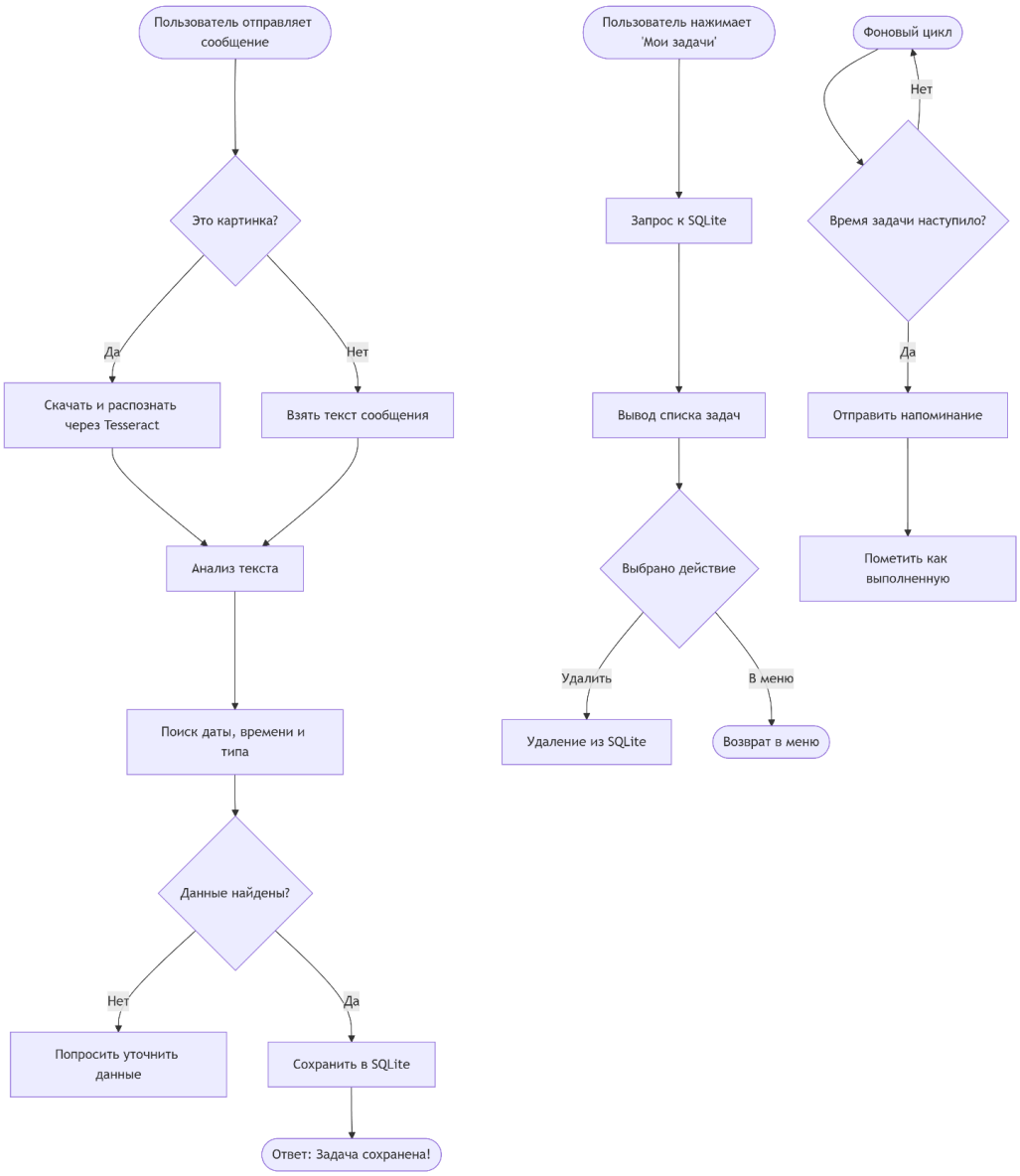
3. Целевая аудитория

- **Школьники 5–11 классов** (нужно не пропускать дедлайны).
- **Учителя** (не терять отчеты и поручения).
- **Родители** (контролировать учебный процесс).

4. Техническая реализация

- **Язык:** Python 3.11.
- **Библиотека для MAX:** `maxapi`.
- **База данных:** SQLite (локальное хранение задач).
- **Распознавание картинок (OCR):** Tesseract OCR (русский + английский).
- **Парсинг дат:** Собственный алгоритм на основе `datetime`.
- **Контейнеризация:** Docker.

5. Логика работы бота (Блок-схемы)



Сценарий 1: Добавление задачи

1. **Старт.** Пользователь пересылает боту сообщение или картинку.
2. **Условие:** Это картинка?

- *Да:* Бот скачивает картинку и отправляет её в Tesseract OCR. Получает текст.
- *Нет:* Бот берет текст из сообщения.

3. **Анализ текста:** Бот ищет дату, время и определяет тип задачи (ДЗ, олимпиада и т.д.).
4. **Условие:** Найдена ли дата и время?

- *Нет:* Бот просит пользователя уточнить данные.
- *Да:* Бот сохраняет задачу в SQLite.

5. **Ответ.** Бот присылает подтверждение: «☑ Сохранил: [Дата] в [Время]».

Сценарий 2: Просмотр и управление задачами (Удаление)

1. Пользователь нажимает кнопку «📋 Мои задачи».
2. Бот выводит список активных задач.
3. Пользователь нажимает кнопку «🗑 Удалить».
4. Бот предлагает варианты удаления:

- *Удалить по номеру:* пользователь вводит номер задачи (например, 1).
- *Удалить по дате:* пользователь вводит диапазон дат.
- *Удалить всё:* бот просит подтверждение и очищает базу данных для этого пользователя.

5. Бот удаляет запись из базы и обновляет список.

Сценарий 3: Фоновое напоминание

1. Фоновый цикл бота раз в секунду проверяет базу данных.
2. **Условие:** Есть ли задачи, время которых наступило?

- *Да:* Бот отправляет сообщение («🔔 НАПОМИНАНИЕ!»), помечает задачу выполненной и убирает её из активного списка.

6. Ограничения текущей версии (MVP)

Наш бот — это **рабочий прототип**. Для победы в хакатоне мы реализовали ключевую механику, но честно оцениваем ограничения:

1. **Только текст и картинки:** Бот не понимает аудио, видео и документы (PDF, Word).
2. **Только календарные события:** Бот умеет парсить даты, но не структурирует записи без привязки ко времени (например, просто списки книг).
3. **Нет хранения контента:** Бот сохраняет только распознанный *текст* задачи. Если вы переслали файл (например, конспект), бот не сохранит сам файл, чтобы вы могли открыть его позже.
4. **Локальная база:** SQLite подходит для прототипа, но при тысячах пользователей потребуется серверная СУБД.

7. Масштабирование и развитие (Future Vision)

Для создания полноценной системы в будущем мы планируем:

1. **Расширение медиа-возможностей:** Добавить хранение файлов (облачное хранилище S3). Бот сможет не только напоминать о контрольной, но и прикреплять к ней файл с правилами или конспект.
2. **Умный структуратор записей:** Выйти за рамки календаря. Например, пользователь фотографирует обложку любимой песни или книгу — бот распознает, определяет тип «Музыка/Книга», сохраняет в отдельный плейлист/список и присылает ссылку на прослушивание.
3. **Интеграция с другими платформами:** Подключение к VK и Telegram.
4. **Улучшение NLP:** Переход от жестких правил к легковесным нейросетям для более точного понимания контекста.

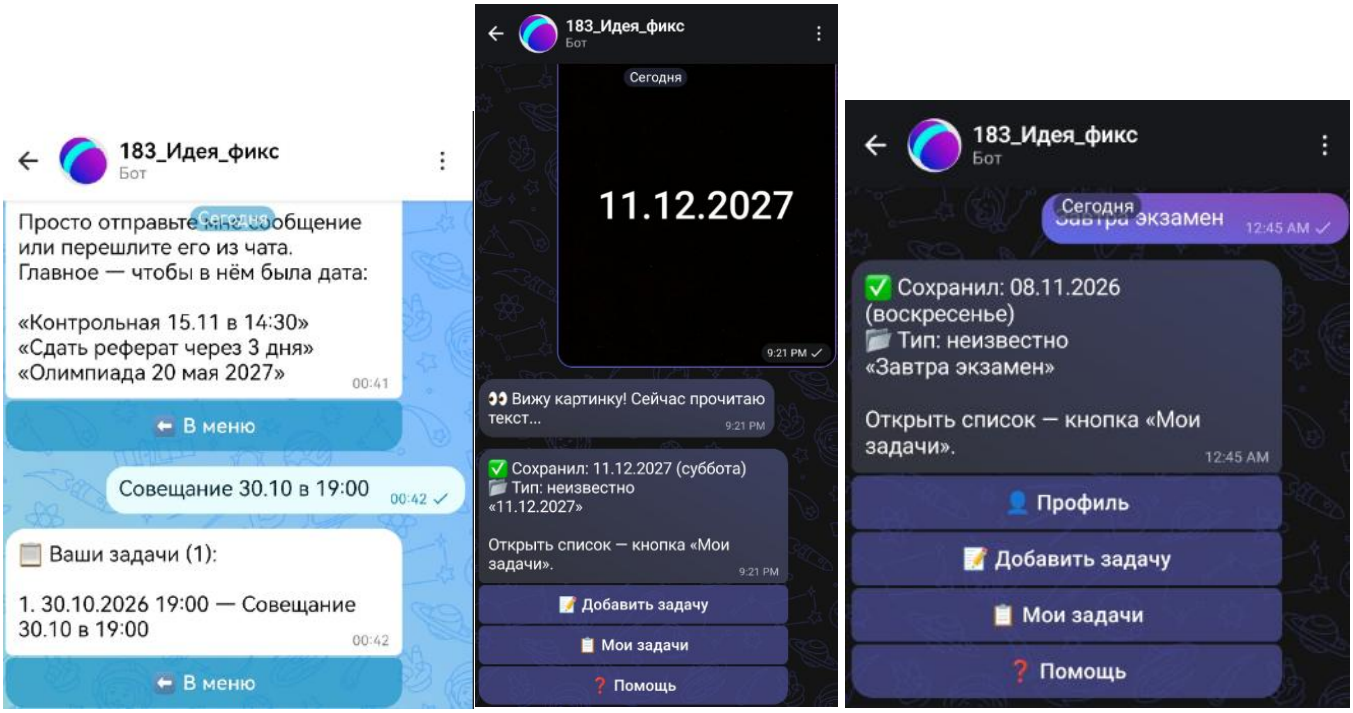
8. Оценка ресурсов для полноценной версии

Создание полноценной версии (не прототипа) потребует следующих ресурсов:

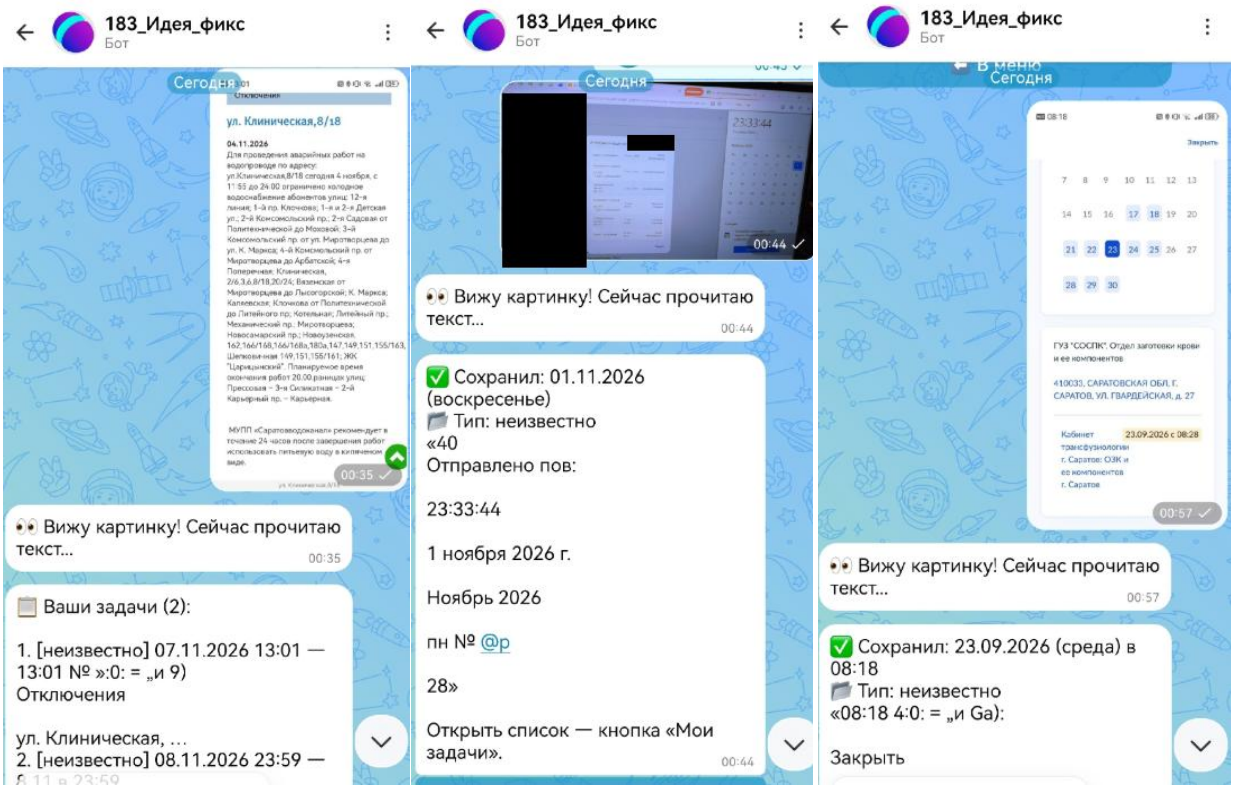
- **Команда (5-7 человек):** 2 Backend-разработчика (Python), 1 Frontend-разработчик (для веб-панели управления и мини-приложения), 1 ML-инженер (для парсинга и работы с LLM), 1 UI/UX дизайнер, 1 QA-инженер.
- **Время:** От 3 до 6 месяцев разработки.
- **Технические ресурсы:** Аренда облачного сервера (VPS), база данных PostgreSQL, объектное хранилище (Yandex Object Storage или S3) для файлов, оплата API нейросетей (OpenAI или YandexGPT).

9. Примеры работы

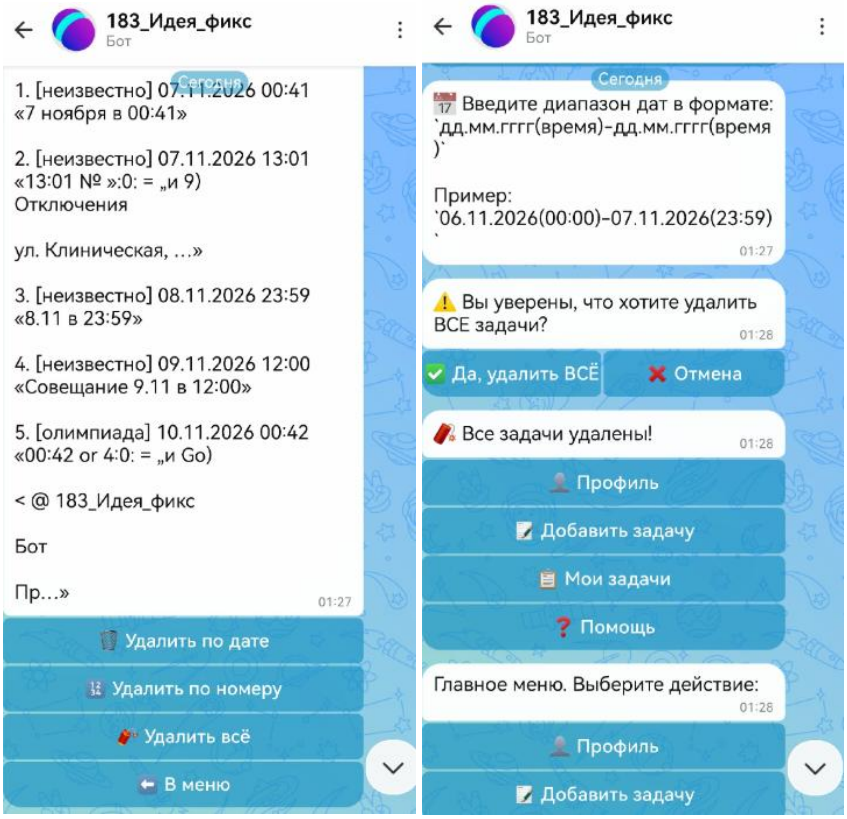
- **Добавление текстового события**



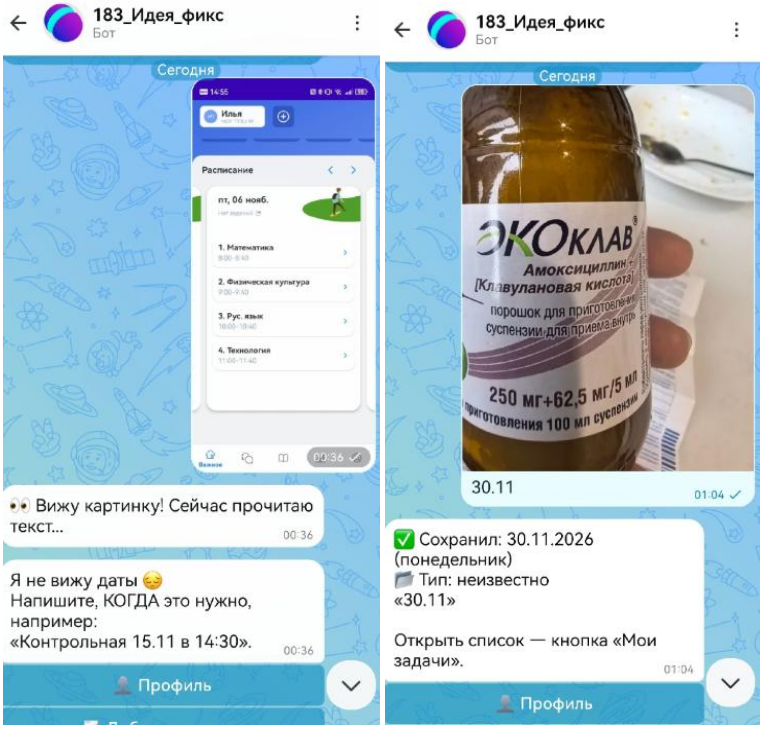
- **Добавление событий с картинки**



- **Удаление**



Проблемные



Даты «30 нояб» и картинка вместе текстом пока не обрабатывается.

- **Итог**

